Class: Algorithms & Data Structures Semester: Spring 2025

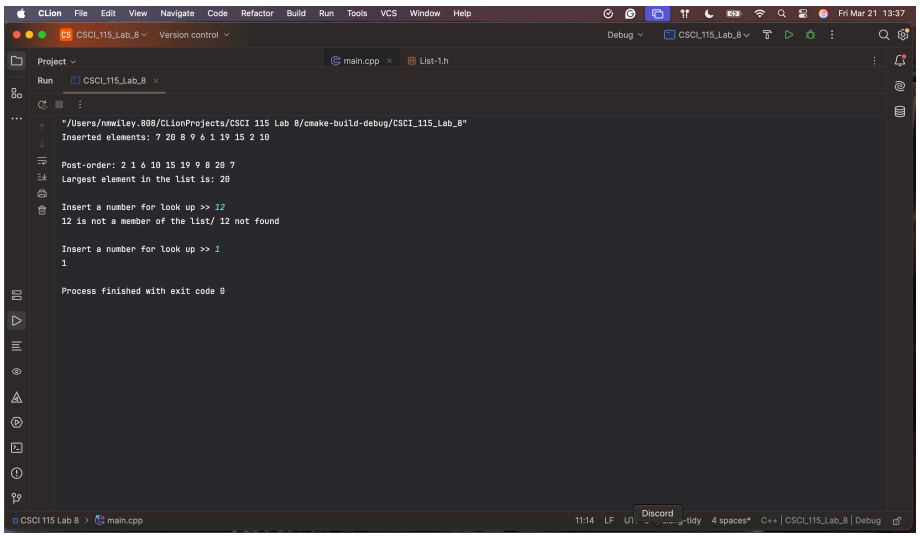
Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

CSCI 115 Lab 8 Report; Binary Search Tree

- Statement Of Objective
  - The objective of this lab is to gain hands-on experience with implementing and using a Binary Search Tree (BST) in C++. The lab focuses on essential BST operations such as insertion , searching, post-order traversal, and retrieving the largest element in the tree. By completing this lab, we aim to understand the internal mechanics of BSTs, how recursive logic applies in tree structures, and how search efficiency is maintained with logarithmic time complexity (assuming balanced trees). The significance of this lab lies in reinforcing recursive problem-solving skills and understanding the behavior and utility of hierarchical data structures. This report details the experimental procedure, analysis, encountered challenges, and conclusions drawn from this experiment.

- Experimental Procedure
  - Created a class BST with a Node structure that holds integer data and pointers to left and right children
  - Implemented a constructor that initializes an empty tree
  - Developed a recursive InsertNode function to insert integers into the tree, ensuring no duplicates
  - Created a public Insert function to encapsulate node insertion and update the root
  - Implemented printTree for post-order traversal
  - Adding a Largest function to find the maximum value in the BST
  - Created a FindNode function to search for elements in the tree
  - Write a main program to insert values, print the tree post-order, retrieve the largest value, and search for user-input values.
  - Validated the implementation against the expected sample output

- Analysis
  - The implementation produced the following results:



  - These results confirm that:
    - The tree structure is built correctly through recursive insertion
    - The post-order traversal works as expected
    - The largest element (rightmost node) is identified correctly
    - The search function correctly distinguishes between present and absent values

- Encountered Problems
  - Initially encountered a complier error: "Cannot assign to rvalue" when trying to assign to tree.getRoot(). This was resolved by modifying the class to include a Insert function that updates the root internally. So I made some changes to the Header File that was given so I would work on my Coding Software I use (CLion).
  - Ensured that the tree ignored duplicate insertions by handling equal values properly
  - Debugged post-order traversal by checking left and right subtree transversal order
  - General debugging involved stepping through recursive calls and visualizing tree structure

- Conclusions
  - Through the lab, the following were learned:
    - How to implement a Binary Search Tree and use recursion effectively
    - Importance of maintaining tree structure for efficient operations
    - Post-order traversal and how it differs from in-order and pre-order
    - How to debug tree-based recursive functions
    - Proper encapsulation of functionality in class design

- References

- Lab 8 - Binary Search Tree Slides